

ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.



**Assignment : Designing an Optimal Disk Management
Scheme for a Hypothetical Operating System
Operating Systems : CSE 4501**

<u>Student ID: 210042112</u>

Fifth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

April 29, 2025

Abstract

Imagine you are tasked with designing the disk management module for a new operating system targeted at high-performance computing (HPC) environments. Your design must be informed by the principles and trade-offs discussed in your textbook.

Contents

1	Disk Performance Requirements for HPC Environments	4
1.1	Throughput	4
1.2	Latency	4
1.3	Reliability	4
1.4	Scalability	4
1.5	I/o Parallelism	4
2	Disk Scheduling Strategy	4
2.1	Rationale for Choosing C-SCAN:	4
2.2	Modifications for HPC	5
2.3	Comparison with Alternative Algorithms	5
3	Disk Formatting, Space Management, and Data Reliability	5
3.1	Disk Formatting	5
3.2	Space Management	6
3.3	Data Reliability	6
4	Theoretical Justifications and Trade-Offs	6

1 Disk Performance Requirements for HPC Environments

1.1 Throughput

HPC workloads, such as large-scale simulations or big data analytics, require high data transfer rates to handle massive datasets. Throughput must be maximized to support parallel I/O operations across multiple nodes, often exceeding tens of GB/s in modern HPC clusters.

1.2 Latency

Low latency is critical for time-sensitive applications like real-time scientific computations. Seek time and rotational latency must be minimized to ensure rapid access to data, ideally in the range of microseconds for SSDs or milliseconds for HDDs.

1.3 Reliability

Data integrity is paramount in HPC, where computations may run for days or weeks. Disk failures must not compromise results, necessitating robust error detection, correction, and fault tolerance mechanisms.

1.4 Scalability

The disk system must support large-scale storage (petabytes) and handle concurrent I/O requests from thousands of processes efficiently.

1.5 I/o Parallelism

HPC applications often use parallel file systems (e.g., Lustre, GPFS) to distribute data across multiple disks, requiring the disk management module to optimize for parallel access patterns.

2 Disk Scheduling Strategy

Chosen algorithm:C-SCAN (Circular SCAN) Based on the HPC requirements, I propose the C-SCAN disk scheduling algorithm, with modifications to leverage modern storage technologies like SSDs and parallel file systems.

2.1 Rationale for Choosing C-SCAN:

- **High Throughput:** C-SCAN processes requests in a circular manner, moving the disk head in one direction (e.g., inward) and servicing all requests in order before jumping to the outermost track and repeating. This reduces seek time variance compared to algorithms like SSTF, ensuring consistent throughput for parallel I/O workloads.
- **Fairness and Predictability:** HPC environments often involve multiple processes issuing I/O requests concurrently. C-SCAN's systematic approach prevents starvation (unlike SSTF, where requests far from the current head position may be delayed indefinitely) and provides predictable latency, critical for time-sensitive HPC tasks.
- **Compatibility with Parallel File Systems:** C-SCAN aligns well with striped data access patterns in parallel file systems, where data is distributed across multiple disks. Its predictable head movement complements the coordinated I/O scheduling required in such systems.
- **Adaptability to SSDs:** While C-SCAN was designed for HDDs, it can be adapted for SSDs (common in HPC) by prioritizing logical block address (LBA) ordering over physical head movement, reducing latency in flash-based storage.

2.2 Modifications for HPC

- **Request Batching:** To maximize throughput, batch I/O requests from multiple processes into a single C-SCAN cycle, reducing overhead and leveraging the high queue depth of modern NVMe SSDs.
- **Priority Queues:** Introduce priority levels for critical I/O requests (e.g., checkpointing data) to ensure low latency for time-sensitive tasks.
- **Parallel Scheduling:** For multi-disk systems, implement C-SCAN independently on each disk in a RAID or parallel file system, coordinating schedules to avoid contention at the controller level.

2.3 Comparison with Alternative Algorithms

- **FCFS (First-Come, First-Serve):** FCFS is simple but leads to excessive seek times and poor throughput in HPC due to random head movements. It's unsuitable for high-concurrency workloads.
- **SSTF (Shortest Seek Time First):** SSTF minimizes seek time for individual requests but can cause starvation and unpredictable latency in high-load scenarios, making it less reliable for parallel I/O.
- **SCAN:** SCAN moves the head back and forth across the disk, servicing requests in both directions. While effective, it introduces higher latency variance than C-SCAN due to bidirectional movement, which can disrupt predictable throughput in HPC.
- **LOOK and C-LOOK:** These are variants of SCAN and C-SCAN that stop at the last request rather than the disk's edge. While slightly more efficient for sparse request distributions, the benefits are marginal in HPC, where request queues are typically dense due to high concurrency.

C-SCAN's balance of throughput, fairness, and predictability makes it the best fit for HPC, especially when adapted for modern storage technologies.

3 Disk Formatting, Space Management, and Data Reliability

3.1 Disk Formatting

- **Low-Level Formatting:** For HDDs, perform low-level formatting to divide the disk into sectors with error-correcting codes (ECC). For SSDs, rely on the controller's built-in formatting to optimize flash memory organization. This ensures compatibility with high-performance storage devices.
- **Partitioning:** Use a single partition per disk to simplify management in HPC, where parallel file systems like Lustre handle data distribution across nodes. Each partition is formatted with a high-performance file system (e.g., XFS or ZFS) to support large files and high I/O rates.
- **File System Choice:** Adopt XFS for its scalability, low-latency metadata operations, and support for large files (common in HPC). XFS's allocation group design minimizes contention in parallel I/O.

Why not alternatives?

- **ext4:** While reliable, ext4 has lower scalability for large-scale HPC workloads compared to XFS.
- **NTFS:** Designed for Windows, NTFS is less optimized for HPC's parallel I/O patterns.
- **ZFS:** Offers robust reliability but introduces higher overhead due to its copy-on-write mechanism, which may degrade performance for write-intensive HPC workloads.

3.2 Space Management

- **Block Allocation:** Use extent-based allocation (supported by XFS) to reduce fragmentation and improve sequential read/write performance for large files. Extents minimize metadata overhead compared to block-based allocation, critical for HPC’s massive datasets.
- **Free Space Management:** Implement a B-tree-based free space list to efficiently track available extents, enabling fast allocation for large files.
- **Caching:** Leverage a write-back cache with asynchronous flushing to reduce latency for write-heavy workloads, such as checkpointing in simulations. The cache is tuned to flush large sequential writes, matching HPC access patterns.

Why not alternatives?

- **Bitmap-based Free Space:** Bitmaps are simpler but scale poorly for petabyte-sized file systems, leading to slower allocation.
- **Write-Through Cache:** Write-through caching ensures immediate durability but increases latency, unsuitable for HPC’s performance demands.

3.3 Data Reliability

- **Error Detection and Correction:** Use ECC at the hardware level (for HDDs) and CRC checksums at the file system level (XFS) to detect and correct data corruption.
- **Redundancy:** Implement RAID-6 across multiple disks to tolerate two simultaneous disk failures, critical for long-running HPC computations. RAID-6 balances reliability and storage efficiency, unlike RAID-5 (single failure tolerance) or RAID-1 (high storage overhead).
- **Backup and Recovery:** Integrate with a parallel file system’s snapshot mechanism (e.g., XFS snapshots) to create point-in-time backups for recovery from logical errors. Periodic checkpoints of critical data are stored on separate storage nodes to ensure fault tolerance.
- **Wear Leveling for SSDs:** For SSD-based storage, rely on the controller’s wear-leveling algorithms to extend device lifespan, supplemented by file system optimizations to minimize write amplification.

Why not alternatives

- **RAID-5:** Less reliable than RAID-6, as it tolerates only one disk failure, risking data loss in large HPC clusters.
- **No RAID:** Unacceptable due to the high risk of data loss in long-running computations.
- **Tape Backups:** Too slow for HPC’s rapid recovery needs, despite high capacity.

4 Theoretical Justifications and Trade-Offs

- **Performance vs. Complexity:** C-SCAN with batching and priority queues increases complexity but maximizes throughput and fairness, critical for HPC. Simpler algorithms like FCFS sacrifice performance.
- **Scalability vs. Overhead:** XFS and extent-based allocation add metadata overhead but scale better for large files and parallel I/O.
- **Reliability vs. Cost:** RAID-6 and snapshots increase storage and computational costs but ensure data integrity, a non-negotiable requirement in HPC.

- **SSDs vs. HDDs:** The design prioritizes SSD compatibility due to their lower latency and higher throughput.

The proposed disk management module for an HPC operating system leverages C-SCAN scheduling, XFS file system, extent-based allocation, RAID-6, and robust error-handling mechanisms to meet the demands of high throughput, low latency, and uncompromising reliability. Each design choice is grounded in balancing performance, scalability, and fault tolerance while addressing the unique challenges of HPC environments.